

Incentivizing Mobility of Multi-agent Vehicle Swarms with Deep Reinforcement Learning for Sensing Coverage Optimization

Mohib Azam
mohib.azam@stonybrook.edu
Department of Computer Science,
Stony Brook University
Stony Brook, NY, USA

Xinlei Chen
chen.xinlei@sz.tsinghua.edu.cn
Shenzhen International Graduate
School, Tsinghua University
Shenzhen, China
Peng Cheng Laboratory
Shenzhen, China

Susu Xu
susu.xu@stonybrook.edu
Department of Computer Science,
Stony Brook University
Stony Brook, NY, USA

ABSTRACT

Mobile Crowdsensing Systems (MCS) often utilize non-dedicated mobile agents (e.g., ridesharing cars) equipped with sensors to collect urban data as they move around a city. Utilizing these non-dedicated agents provides a cheap means for mobile crowdsensing. However, driver agents often focus on traveling to areas where they can pick up more passengers, which can cause the distribution of uploaded data to mismatch our intended distribution. In this work, we propose a passive incentivization scheme in which incentives are applied to different map locations for agents who travel to that location at that time and upload data. We design a simulated environment for multi-agent mobile crowdsensing systems based on real-world taxi data and model large amount of agents with heterogeneous utility functions and routing behaviors, namely myopic, semi-myopic, and farsighted. With this environment, we propose using reinforcement learning to search for an ideal incentivization policy. Finally, we show through experiments that a Proximal Policy Optimization-based method is capable of learning such a policy in spite of the complexities and high dimensionality of the environment and action spaces.

CCS CONCEPTS

• Human-centered computing → Mobile computing; • Computing methodologies → Mobile agents; • Information systems → Sensor networks.

KEYWORDS

Mobile Crowd Sensing, Reinforcement Learning, Sensing Quality Optimization, Combinatorial Optimization

ACM Reference Format:

Mohib Azam, Xinlei Chen, and Susu Xu. 2022. Incentivizing Mobility of Multi-agent Vehicle Swarms with Deep Reinforcement Learning for Sensing Coverage Optimization. In *Proceedings of the 2022 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp/ISWC '22 Adjunct)*, September 11–15, 2022, Cambridge, United Kingdom.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

UbiComp/ISWC '22 Adjunct, September 11–15, 2022, Cambridge, United Kingdom

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9423-9/22/09...\$15.00

<https://doi.org/10.1145/3544793.3560410>

Adjunct), September 11–15, 2022, Cambridge, United Kingdom. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3544793.3560410>

1 INTRODUCTION

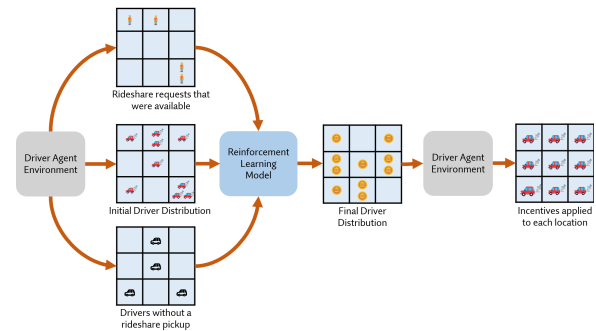


Figure 1: The framework overview of our multi-agent crowdsensing system. In this system, multiple agents are running with their own heterogeneous utility functions, considering traffic conditions, ride request distributions and incentives from the crowdsensing platform. Our final goal is to incentivize some of the agents by providing an incentive map, such that their location distribution matches a given objective distribution. If an agent arrives at one location during a specified time slot, the agent will get the incentive value listed for that location. This is a passive incentive design. In the example shown in the figure, our objective distribution for the data collection is a uniform distribution.

The proliferation of mobile multi-agent systems, such as ridesharing platforms (Uber, Lyft, and traditional taxis) and drone platforms (swarm drone delivery systems), allows for a cheap method for non-dedicated mobile crowdsensing. By deploying sensors on these existing mobile agents, various urban data, such as air pollution and traffic conditions, can be collected as agents move inside a city [2, 8]. However, one major challenge associated with this non-dedicated mobile sensing platform is that the agent distribution may significantly differ from the objective distribution for crowdsensing. This is mainly because driver agents aim to make as much money as possible, thus they will naturally travel most through areas that may have the most passengers they can collect fares from at a given time. As a result, the amount and quality of sensing data from unpopular areas will be largely constrained, which can

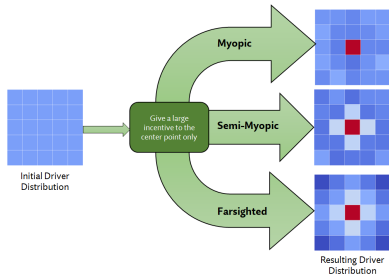


Figure 2: The heatmaps showcase the driver agent distribution at different timesteps, with hotter colors representing a larger number of agents at a location. We consider a driver agent distribution where all drivers are initially distributed uniformly and a particularly large incentive to the center of the map and no incentives anywhere else. At the following timestep, if all of the drivers are myopic, the drivers adjacent to the center will move into it, while all other drivers will ignore it. If the drivers are semi-myopic, then the drivers within two spaces of the reward will start to move towards it at the next timestep. Meanwhile, if all drivers are farsighted, then all agents will be within range of seeing the center point, so they all begin to move towards it.

result in an unfair data distribution over disadvantaged areas and inefficient data collection.

In order to achieve a desirable distribution, one straightforward solution is to provide incentives for driver agents (i.e. money) to travel to locations that are expected to have insufficient data otherwise, which will also dissuade drivers from visiting highly populated areas. We have previously explored doing this by selectively picking and incentivizing partial vehicle agents in an active manner [1, 15, 16]. In the active solution, incentivized agents will be provided with alternative routes associated with incentives that compensate the utility reductions compared to their expected trajectory [14, 16]. When incentivized agents follow the new optimized routes, the overall agent distribution will approximate the objective distribution. However, generating alternative routes with incentives for each driver agent is computationally expensive, if not impossible, when a large number of drivers (more than one hundred) exist in a map. Doing so would require us to directly learn a probabilistic model for each agent’s actions to predict where the agent would travel otherwise.

Our contributions in this paper are as follows. Firstly, instead of directly predicting agent behavior and using that to create alternative trajectories, we propose a simpler approach—passive incentivizing. Each location on a map is assigned a monetary value, and this value is given to any driver agent that goes to that location and uploads data for it at the next timestep. The incentives should be organized in such a way that they passively influence driver agent decisions so that the driver agent distribution more closely matches our objective.

Secondly, we provide an environment for our problem with heterogeneous agent behavior. Our environment contains multiple types of agents—myopic, semi-myopic, and farsighted—differentiated by their ability to predict their future utilities. These agents move

around a map area, with their actions being stochastically determined based on how much money they expect to obtain, considering both the likelihood of a passenger pickup and any incentive provided. Additionally, the ride request data at each timestep is based on real-world data.

Finally, we show that Reinforcement Learning (RL) is a viable way to approach this problem, as it can more effectively scale to complex environments without needing to make direct assumptions about driver agent behavior. We found that policy-based methods, specifically a Proximal Policy Optimization (PPO)-based passive incentivizing method, perform the best [10]. This is due to PPO’s clipped objective function, which allows for stable training with strong convergence guarantees, as well as its ability handle continuous action spaces as opposed to discrete action spaces.

The rest of the paper is organized as follows. In Section 2, we present a simulation environment for driver agents within a city, based off of real taxi data. We then, in Section 3, propose a reinforcement learning approach for assigning incentives to different locations on a city map. In Section 4, we provide experimental results showing the viability of RL. In Section 5 we discuss related works. Finally, in our conclusions in Section 6, we discuss potential further extensions for our RL approach.

2 HETEROGENEOUS MULTI-AGENTS CROWDSENSING SIMULATION PLATFORM

In this section, we mainly describe our designed simulation platform for modeling and simulating heterogeneous agents’ reactions to incentives and incentivized mobilities in a mobile crowdsensing platform. To better simulate real-world scenarios, we utilize ride request distributions and trajectories as well as traffic conditions from real-world large-scale datasets collected from more than 30,000 taxis in Beijing, China. Meanwhile, we designed heterogeneous agent behavioral models with varying myopic levels to better represent complex and varied driver reactions to the incentives. In the following subsections, we present our traffic environment simulation and agent behavioral model design. For notation purposes, at timestep t , our environment is defined as having a map represented as a matrix grid with dimensions $L \times W$, a collection of ride requests $R(t)$ for each timestep, and a list of N driver agents.

2.1 Traffic Environment Simulation

Our ride request data is based off of real taxi data [3, 12]. This dataset is derived from GPS data of taxi drivers operating in Beijing during November 2012, November 2014, and November 2015. From the large amount of information in this dataset, we are specifically interested in the times when a taxi is occupied with a passenger, taking into account the latitude and longitude of the taxi as it drives this passenger from a starting location to their destination.

For our purposes, we treat instances of a driver moving with a passenger as a ride request. We discretize each request into 720 timesteps over a 24 hour period, meaning each time point t represents a 2 minute period in real-world time. We remap the resulting latitude and longitude values of a request to fit into a $L \times W$ space by providing latitude and longitude boundaries for the entire map area and then uniformly converting a request’s latitude and longitude locations to a respective grid location. Using this, we now

have a transformed ride request that contains a list of map coordinates (i, j) , where $i, j \in \mathbb{Z}$ and $i \in [0, L - 1]$, $j \in [0, W - 1]$, to visit at each time point t . Ride requests appear at each time point t in the environment. After all agents move, agents at each location are randomly selected to receive a random ride request, with each ride request only getting assigned to one agent. If a request is not assigned to an agent, we do not reuse it for the next timestep; each timestep only contains the requests that started at that time, according to the dataset.

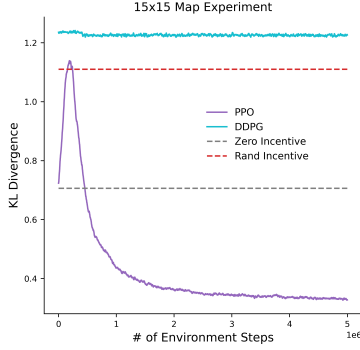


Figure 3: A plot of the objective and driver’s KL-divergence obtained by using PPO and DDPG on an environment with a 15 x 15 map and 1,000 myopic agents. We see that DDPG, while very slightly improving at around 500,000 iterations, fails to make any significant training progress, with the KL-divergence hardly changing and still being above our other baselines. Meanwhile, PPO, despite getting worse at first, later recovered and obtained a significantly lower KL-divergence.

2.2 Heterogeneous Agents Behavioral Model Design

Not all drivers will focus on searching for passengers within their local vicinity. Jiang et al. [3] found that drivers from their collected dataset of taxi drivers in China often traveled longer distances during their search for passengers rather than focusing on finding a passenger nearby. Therefore, while we usually consider myopic drivers that aim to pick up passengers in their local vicinity, it is important to consider drivers that look further ahead. Thus, we introduce semi-myopic and farsighted agents to our environment. Semi-myopic agents take actions based on what path they believe can provide the best utility in the near future, while farsighted agents consider even farther into the future. Fig 2 provides an example where we provide money for any driver agent in the center of the map. Non-adjacent myopic agents can’t immediately tell this money exists and wander the map, semi-myopic agents within a larger proximity of the incentive start to move towards it, and farsighted agents will look ahead far enough to notice this potential incentive and then start to take a path moving towards it. Thus, to be more realistic, our simulator should be able to model these different types of driver agent behaviors.

Each driver agent is represented by its location (i, j) , a value n , which is the number of steps it can look ahead when computing utilities for follow-up actions, and an itinerary containing a list of

locations (p, q) in a ride request that the agent is handling, so the agent will be guaranteed to move to those points at each of the next timesteps. If this itinerary is empty, then the driver agent is free to be assigned a new request. At each timestep of the environment t , an Incentive $I(i, j)$ is applied to each point (i, j) on the environment map as an action. At the beginning of each timestep, driver agents with a request will move to the next location designated in their request. Therefore, drivers that are fulfilling a ride request are unaffected by incentives. Drivers who do not have a request will instead either move to an adjacent grid location (or stay in their current location). Drivers will calculate the utility of each of these locations as the average n -step non-discounted utility that can be obtained from traveling to that location.

For our experiments, myopic (our defaults) drivers use a 1-step utility, semi-myopic drivers use a 2-step utility, and farsighted drivers use a 5-step utility. While each of our experiments only consider one type of agent, our environment is capable of handling combinations of different agent types and agents with different values of n other than those defined above.

All incentives provide their respective value as a return, and we assume that all requests provide a return of 10 if a passenger is picked up, but no return is given while a request is being handled. Thus, the request utility is 10 multiplied by the probability of finding a passenger, which is calculated based on the number of empty drivers already at that location: $Pr(\text{GettingRequest}) = \frac{|R(i, j, t)|}{E(i, j)}$. The overall utility is calculated as

$$U_{driver}(i, j) = \frac{I(i, j) + (10 * Pr(\text{GettingRequest})) + \max_{(i', j')} (U(i', j'))}{n}$$

, where (i', j') is a possible destination the driver agent can move to from (i, j) at the following timestep.

Realistically, drivers will not necessarily act fully rationally, so we weight every possible action with a softmax distribution over each of the respective action utilities. The action of the driver agent is then randomly chosen using this softmax distribution.

3 PASSIVE MULTI-AGENT INCENTIVIZING PROBLEM FORMULATION

In this section, we define our objective for the driver agent incentivization problem. Following that, we define our problem as an MDP and suggest an RL approach for finding a solution.

3.1 Objective Definition

In a heterogeneous multi-agent crowdsensing system, most agents take actions to maximize their own utilities (e.g., taking more passengers to make more money), and their utilities are often not directly related to our mobile crowdsensing goals. Therefore, our driver agent distribution D likely differs from an ideal distribution that will allow for effective data collection. In this work, our final goal is to best utilize all agents to optimize the sensing coverage quality, such that most important information about the target area can be acquired. Suppose we have an environment that has a map of dimensions $L \times W$ and a number of drivers $N(i, j)$ at each location (i, j) , where $i \in [0, L]$ and $j \in [0, W]$. Let D represent the

distribution of driver agents across our map

$$D(i, j) = \frac{N(i, j)}{\sum_{i,j} N(i, j)} \quad \forall i \in [0, L], \forall j \in [0, W]$$

Let O represent our objective distribution. Following our previous work [16], we choose to use Kullback-Leibler Divergence KL as our metric to evaluate our sensing coverage quality, i.e., how similar the current driver agent distribution D is to our goal O . Our goal is to create a model that provides incentives to the environment at each timestep that influence D in such a way that KL is minimized.

$$KL = \sum_{i,j} O(i, j) \log \frac{O(i, j)}{D(i, j)}$$

3.2 State, Action, and Reward Design for Reinforcement Learning

In this section, we mainly describe the design of states, actions, and rewards in our environment, which are the key components for reinforcement learning. Because our environment's states at each timestep are only directly based off of the state from the previous timestep, we can model our problem as a Markov Decision Process (MDP) that can be solved via reinforcement learning. With the designed state, action, and reward, we utilize Proximal Policy Optimization (PPO) to create a neural network-based policy for our dynamically changing traffic environment containing complex variations of driver agent behaviors.

3.2.1 State Representation. The designed states are expected to represent as much information of the dynamically changing traffic environment and complex variations of agents behaviors as possible. Let a state S be defined as an $L \times W \times 3$ array stacking together three different channels of information. The first channel contains the number of ride requests $R(i, j, t)$ that were at each location for that timestep, regardless of whether or not that request was accepted by an agent. The second channel contains the number of driver agents at each location $N(i, j)$, and the third channel contains the number of driver agents at each location that are not currently occupied with a request $E(i, j)$.

3.2.2 Action. Our action space is an $L \times W$ matrix containing an incentive to be applied to each point (i, j) on the map. These incentives are continuous values within a range $[0, I]$, where I is the environment's pre-defined maximum incentive value that can be assigned to a location. Using a continuous action space is much preferred over a discrete action space for this environment due to the flexibility in incentive assignment it provides. Additionally, a discrete action space based on this environment would be prohibitively large: For an $L \times W$ map with actions discretized as integers in range $[0, I]$, there would be $I + 1$ possible incentives for each location (i, j) , giving a total action space of $(I + 1)^{L \times W}$.

3.2.3 Reward Design. As discussed in Section 3.1, we aim for our RL model to minimize the KL -divergence between our objective distribution O and our driver agent distribution D . This is a submodular combinatorial optimization problem, as the agent distribution is a relative measurement calculated from comparing different locations. However, RL methods are designed to find a maximum reward, and rewards are typically normalized to be within a range

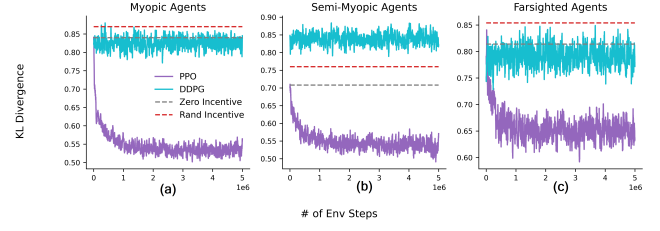


Figure 4: A plot PPO and DDPG being trained on a 5×5 map with all of agents either being myopic (a), semi-myopic (b), or farsighted (c). We see that PPO obtains similar KL -divergence values for both myopic and semi-myopic agents, but performs slightly worse for farsighted agents. DDPG fails to achieve noticeable improvements for any of them.

$[-1, 1]$ to allow for stable and converging training. As such, we will transform KL to fit in this range such that low KL gives a positive reward and high KL gives a negative reward. For a given threshold value T , where T is the largest KL allowed before the reward gets clipped to -1 (avoiding a long-tailed distribution), the reinforcement learning utility is calculated as

$$U = \max \left(\frac{((T/2) - KL)}{(T/2)}, -1 \right)$$

3.2.4 Proximal Policy Optimization for Real-time Large-scale Incentive Generation. Proximal Policy Optimization (PPO) [10] is an on-policy reinforcement learning model that trains a stochastic policy. PPO is based on the actor-critic model, in which a policy network (actor) π_θ is trained alongside a value network (critic) V . The goal of the critic is to accurately predict the utility $V(s)$ of any given state s , while the actor aims to create a policy such that the action a it returns has the largest improvement possible over the critic $Q(s, \pi_\theta(a)) - V(s)$. Despite PPO being on-policy, it is capable of exploring the environment thanks to the policy it generates being stochastic, which is very important as our environment contains a large state space that needs to be explored effectively to find a good incentivization policy. There are two key distinctions between PPO and similar methods in Deep Q-Network (DQN) [7] and Deep Deterministic Policy Gradient (DDPG) [4] that make it ideal for our environment. Firstly, a method like DQN is only capable of handling discrete action spaces, since it directly predicts state-action value $Q(s, a)$ for a given state s . This means that its output layer is as large as the action space, prohibiting it from handling continuous action spaces. PPO meanwhile is capable of handling continuous actions thanks to directly returning an action for a given state. Secondly, unlike both DQN and DDPG (another actor-critic method that can handle continuous actions), PPO is designed to have strong convergence properties. This is thanks to it bounding the loss of θ with a clipping parameter ϵ , with the loss being measured as the ratio between the proposed updated policy $\pi_{\theta'}$ and the previous policy π_θ . This way, large updates to the policy that could cause the policy to diverge will not occur, a property that is highly desirable for our complex environment.

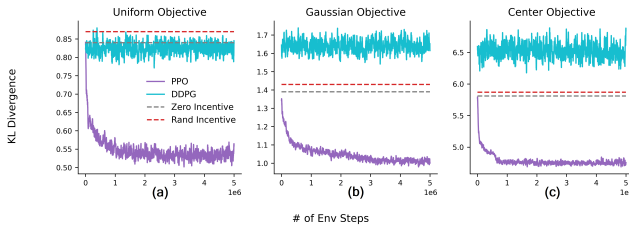


Figure 5: A plot of the performance of PPO and DDPG on the uniform (a), Gaussian (b), and center (c) objectives. We see that PPO is able to reduce its KL -divergence across all objectives as it trains. However, its final performance for a Gaussian objective is worse than that of the uniform objective, and the final performance on the center objective is particularly poor. With non-uniform objectives, DDPG performs worse than zero reward and random reward baselines.

4 EXPERIMENTS

In our experiments, we compare PPO with baseline methods, namely another policy-gradient method in DDPG, across several different environment setups. As mentioned in Section 3.2.3, all experiments were run using normalized rewards in range $[-1, 1]$ to allow for stable training. Our experiments also used normalized actions—the actions returned by the model are in range $[-1, 1]$ and then transformed to be in range $[0, I]$ using $A_{transformed} = (A_{model} + 1) * (I/2)$. All models are constructed as a Multi-Layer Perception (MLP). For the ride request data, we use the real-world data collected by more than 30,000 taxis in Beijing City, China [3]. We specifically use the data from November 2nd, 2015 and start our environment at 8:00 AM, which is peak hour and has large traffic volume. We train our models for 5,000,000 environment steps, with each episode lasting 20 environment steps, giving us the time range of 8:00-8:40 AM. All experiments use a maximum incentive of $I = 10$ and a reward threshold of $T = 2$, with the exception of experiments for the center objective, which instead use $T = 6$ due to high KL -divergence values being much more likely. With the exception of the Gaussian and center objective experiments in Section 4.3, all experiments use a uniform objective; that is, one where all spaces on the map should be occupied by the same number of driver agents.

We use the Stable-Baselines3 implementations of DDPG and PPO [9].

4.1 Large Map Experiments

We test our model’s ability to scale and handle larger map sizes. We use a 15×15 map instead of a 5×5 map. This also requires us to increase the number of agents to 1,000 in order for each grid location to have at least 1 driver in it. These results are shown in Fig 3. DDPG fails to find a reasonable policy; it does manage to slightly improve its KL -divergence after around 500,000 iterations, but then fails to improve further. Interestingly, PPO’s performance actually starts to get continuously worse, but it then starts to substantially improve and not only beat both baselines, but also obtain a KL -divergence lower than that of the 5×5 map experiment.

4.2 Driver Type Experiments

We test PPO and DDPG on environments where all of the agents are semi-myopic or all of the agents are farsighted. We once again use a 5×5 map size with 100 agents. Our results are shown in Fig 4. The final KL -divergence obtained by PPO seems to be slightly higher for both semi-myopic and farsighted agents, and the increased noise of the plot’s tails for farsighted agents indicates that the obtained KL -divergence is not very consistent. Once again, PPO clearly outperforms DDPG for these experiments.

4.3 Objective Experiments

We run our models on a 5×5 map with 100 myopic agents using three different types of objectives. Firstly, there is our standard uniform objective. Secondly, we try a Gaussian objective, where agents should be spread out in a manner similar to a 2D Gaussian distribution, with less agents towards the edges and more agents towards the center of the map. Finally, we use a center objective, where every agent should move to the grid location in the center of the map. Our results are shown in Fig 5. While PPO is still able to improve over all of these objectives, it still does not perform particularly well for the Gaussian objective. It also performs quite poorly for the center objective. The Gaussian objective seems to take the longest time to converge, while the center objective takes the shortest time. However, PPO still clearly outperforms DDPG.

5 RELATED WORKS

Xiao et al. [13] considered the problem of malicious mobile users providing fake results to a mobile crowd sensing server and used a DQN to train a policy for incentivizing users, making sure to reward users based on the information gain of their provided data. Zhan et al. [17] worked on a similar problem and used PPO instead. Liu et al. [5] used a Multi-Agent Deep Deterministic Policy Gradient (MADDPG) model [6] to navigate different vehicles across a map to maximize the amount of total data collected while staying within an energy usage constraint. Finally, Wang et al. [11] focused on sparse mobile crowdsensing and used a Recurrent-DQN to select a minimal number of areas on a large map that are needed for the total sensed data collected to reach some given quality level.

6 CONCLUSION

We have presented an environment for simulating mobile agents with incentivization. We have shown that reinforcement learning is a viable approach for producing incentives for each location in such a way that the driver agent distribution can match a given objective distribution in order to achieve a proper sensor distribution for mobile crowdsensing. For future work, we aim to extend our map representation to be a graph instead of a matrix, allowing us to form edges between non-adjacent grid locations on a map. For example, we noticed that some requests in our data had driver agents travel between two grid locations that were more than 3 units away from each other, suggesting that the map is connected in a non-adjacent fashion and must be more accurately represented as a graph rather than a matrix. We also plan to extend this environment to support other applications of mobile agent systems.

REFERENCES

- [1] Xinlei Chen, Susu Xu, Jun Han, Haohao Fu, Xidong Pi, Carlee Joe-Wong, Yong Li, Lin Zhang, Hae Young Noh, and Pei Zhang. 2020. Pas: Prediction-based actuation system for city-scale ridesharing vehicular mobile crowdsensing. *IEEE Internet of Things Journal* 7, 5 (2020), 3719–3734.
- [2] Bret Hull, Vladimir Bychkovsky, Yang Zhang, Kevin Chen, Michel Goraczko, Allen Miu, Eugene Shih, Hari Balakrishnan, and Samuel Madden. 2006. CarTel: A Distributed Mobile Sensor Computing System. In *Proceedings of the 4th International Conference on Embedded Networked Sensor Systems* (Boulder, Colorado, USA) (*SenSys '06*). Association for Computing Machinery, New York, NY, USA, 125–138. <https://doi.org/10.1145/1182807.1182821>
- [3] Weiwei Jiang, Jing Lian, Max Shen, and Lin Zhang. 2017. A multi-period analysis of taxi drivers' behaviors based on GPS trajectories. In *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*. 1–6. <https://doi.org/10.1109/ITSC.2017.8317622>
- [4] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2016. Continuous control with deep reinforcement learning. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). <http://arxiv.org/abs/1509.02971>
- [5] Chi Harold Liu, Zheyu Chen, and Yufeng Zhan. 2019. Energy-Efficient Distributed Mobile Crowd Sensing: A Deep Learning Approach. *IEEE Journal on Selected Areas in Communications* 37, 6 (2019), 1262–1276. <https://doi.org/10.1109/JSAC.2019.2904353>
- [6] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. 2017. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. *CoRR* abs/1706.02275 (2017). [arXiv:1706.02275](http://arxiv.org/abs/1706.02275) <http://arxiv.org/abs/1706.02275>
- [7] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin A. Riedmiller. 2013. Playing Atari with Deep Reinforcement Learning. *CoRR* abs/1312.5602 (2013). [arXiv:1312.5602](http://arxiv.org/abs/1312.5602) <http://arxiv.org/abs/1312.5602>
- [8] Piyush Patil. 2017. Smart IoT based system for vehicle noise and pollution monitoring. In *2017 International Conference on Trends in Electronics and Informatics (ICEI)*. 322–326. <https://doi.org/10.1109/ICOEI.2017.8300941>
- [9] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* 22, 268 (2021), 1–8. <http://jmlr.org/papers/v22/20-1364.html>
- [10] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *CoRR* abs/1707.06347 (2017). [arXiv:1707.06347](http://arxiv.org/abs/1707.06347) <http://arxiv.org/abs/1707.06347>
- [11] Leye Wang, Wenbin Liu, Daqing Zhang, Yasha Wang, En Wang, and Yongjian Yang. 2018. Cell Selection with Deep Reinforcement Learning in Sparse Mobile Crowdsensing. In *2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*. 1543–1546. <https://doi.org/10.1109/ICDCS.2018.00164>
- [12] Zhengwei Wu, Xiaoxi Zhang, Susu Xu, Xinlei Chen, Pei Zhang, Hae Young Noh, and Carlee Joe-Wong. 2020. A generative simulation platform for multi-agent systems with incentives. In *Adjunct Proceedings of the 2020 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2020 ACM International Symposium on Wearable Computers*. 580–587.
- [13] Liang Xiao, Yanda Li, Guoan Han, Huaiyu Dai, and H. Vincent Poor. 2018. A Secure Mobile Crowdsensing Game With Deep Reinforcement Learning. *IEEE Transactions on Information Forensics and Security* 13, 1 (2018), 35–47. <https://doi.org/10.1109/TIFS.2017.2737968>
- [14] Susu Xu, Xinlei Chen, Xidong Pi, Carlee Joe-Wong, Pei Zhang, and Hae Young Noh. 2019. Incentivizing vehicular crowdsensing system for large scale smart city applications. In *Sensors and Smart Structures Technologies for Civil, Mechanical, and Aerospace Systems 2019*, Vol. 10970. SPIE, 402–408.
- [15] Susu Xu, Xinlei Chen, Xidong Pi, Carlee Joe-Wong, Pei Zhang, and Hae Young Noh. 2019. Vehicle dispatching for sensing coverage optimization in mobile crowdsensing systems. In *2019 18th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*. IEEE, 311–312.
- [16] Susu Xu, Xinlei Chen, Xidong Pi, Carlee Joe-Wong, Pei Zhang, and Hae Young Noh. 2020. iLOCuS: Incentivizing Vehicle Mobility to Optimize Sensing Distribution in Crowd Sensing. *IEEE Transactions on Mobile Computing* 19, 8 (2020), 1831–1847. <https://doi.org/10.1109/TMC.2019.2915838>
- [17] Yufeng Zhan, Yuanqing Xia, Jiang Zhang, Ting Li, and Yu Wang. 2019. Crowdsensing Game with Demand Uncertainties: A Deep Reinforcement Learning Approach. *CoRR* abs/1901.00733 (2019). [arXiv:1901.00733](http://arxiv.org/abs/1901.00733) <http://arxiv.org/abs/1901.00733>